

- Eurostar 2018 Transcriptions
 - Practice Talk Transcript
 - Some Notes and Thoughts on Risk
 - Lightning Talk - Our Agile Process is Screwed Up
 - Lightning Talk Practice with Additional Information about Cybernetics and feedback

Eurostar 2018 Transcriptions

Practice Talk Transcript

This is the practise recording for my Eurostar keynote.

I'm gonna try and give you the full keynote experience 'cause Eurostar asked us to pick a track that we'll use for the intro and the walk on. So I'm gonna play that. Then I'll dive into the talk so you get the full immersive stadium type experience. Alright. So, that was The Trooper.

That was Iron Maiden's, The Trooper. I quite like it. I mean Eurostar want us the pick a song so clearly there has to be a message.

But the reason why I picked it is that there's a common phrase associated with warfare which is "your plan does not survive contact with the enemy". And there's couple of lessons you could draw from that but the lesson I draw from that is we really want our plans to be simple to understand and implement. 'Cause if it's not gonna stand up to contact with the enemy we have to be able to flex around them.

So the more complicated the plan is, the more interdependent it is, the more over complicated stuff is the less we are gonna be able to flex fast when something changes.

The other thing is if it's not gonna survive contact with the enemy, we need to know what the end objective is or multiple end objectives so that we can try and achieve them. And we might even want to know what we're gonna do after we achieve an end objective because if we can't even achieve those objectives we might be able to think of new ways that we might be able to build on it to achieve the ultimate things we gonna do afterwards.

So we've got a chain.

The Trooper is essentially about the person on the ground the soldier that's in there and doing the fighting, doing the job. And what I think is important there particularly for testers is that that person is drilled into them the skill set that they need to handle that environment that they're going in to. In order to be able to flex, in order to even to be able to do the job that they're supposed to do, they have to know basic skills or understanding there. And we don't just throw people on the battlefield and expect them to survive, cause they won't. And then if we expect them to adapt to circumstances, they have to be able to handle things. Even if you were just thrown into a battlefield, you've been given 15 minutes beforehand you're taught how to shoot a gun.

You know the basics, you then have to rely on your common sense. You have to observe what other people are doing. You have to make this work in order to survive.

Now, I have the concern that in testing we over complicate testing right? I think in the industry there's a very big temptation to make it seem really complicated. I mean if you look at some of the definitions that are out there for testing, some of them are so convoluted, and they change.

This is why I find interesting about definitions. I mean if you ever tried to write your own definition, it's hard. But I think it's an exercise that's worth doing. It's a very important exercise to try and define testing. To get in your head what this thing that you're working with is.

We have the issue that if we don't make something that is over complicated. Then when you write your own definition is gonna come out with something over complicated 'cause you're gonna think well it doesn't cover everything that I do. Well that makes it look too easy, that makes that I've over simplified it. I've over simplified it because your writing a definition for yourself and you know how complicated and rich this is.

It's a temptation to put all that in your definition. But if we then make it public, there is a temptation that well then people look at it and go well this is too easy, this is something that we can easily understand. You can go and train yourself as a tester and come out as a certified tester in three days. So this we can just get junior people to do this there's nothing important here. You don't need 15 years experience in order to do this. Then there's a danger in the testing world we might get paid even less. So since we already get paid even less than everyone else we'll get... There's a real danger that we get and paid even less, and we don't want that.

We don't want it to get too simple and too easy therefore we try to cram everything in. Then it's over complicated and that's bad. So when I was thinking about this, I thought right, what I will do, I will create an example. I'll create an example definition of software testing.

Software Testing is a process whereby we use a software system deployed in multiple configurations via its GUI API and all exposed interfaces to gather information about its behaviour, flaws and impact on the user and stakeholder needs and requirements; for the purpose of evaluating the quality and fitness for purpose in specific operational environments and conditions.

Right, 'cause I'm trying to cram in everything. I didn't even try hard to cram in everything. This was me looking at going how would I describe this if I want to cover GUI testing API testing, the different basic feedback, looking at behaviour, impact, all of this stuff. How can I put this in? So it's an over complicated definition.

Now I read that out, I don't remember what it was. I wrote it, I don't remember what it really covers or stands for. It's over complicated. The thing is, even having one of those definitions, we wouldn't stop there. Because we want to continually refine our definition. We want to simplify it because it's too big. But then we want to over complicate it again. So we delete sections, we'll change it over time. Now what's interesting is that we're doing this because we're changing it based on our understanding as it evolves overtime.

We're also doing it because we're communicating right? Now when you're communicate we have a deep structure. We have a model, a very complicated model of what we're trying to communicate, but we have to make it a very simple communication process in order to communicate to different people. And different people understand different things so we communicate in different ways. But this is a communication it isn't what testing is right? 'Cause testing itself probably hasn't changed right? The essence of testing probably hasn't changed over the years. So the definition of testing if it was a description of an absolute, wouldn't have changed over the years.

So we're describing a relationship.

Now I thought, since we over complicate and there's a danger in over complicating. I would look at how other complicated fields describe themselves like physics. Physics is complicated. So I looked in a dictionary, I thought I could go down stairs, get the dictionary but it was too far away so what I'm gonna do, I'm just gonna look for the Google dictionary. 'Cause Google has a dictionary built in, I will ask that for the definition.

And then I get the notion of physics, what is physics?

Physics is the branch of science concerned with the nature and properties of matter and energy.

Whoo! And then it even says the subject matter includes, now what I would prefer almost, is "includes such things as", mechanics, heat, light, structure of atoms, etc. 'cause then it's open ended. If we have open ended definitions we can make them really small. But this, I imagine that physics covers more than mechanics, heat, light, radiation, structure of atoms, stuff like that. But they've limited it there, so people probably argue over that definition but you stick etc. in and you make it open world but bigger it's a little bit easier to communicate to people without arguments.

Now in dictionaries there is very often a derivation which is important right? When we look at software testing, what is a derivation of software testing? We've taken things from math, graph theory, Set theory decision theory, probability theory, we use sampling processes when we're coming up with tests. And particularly Set theory, leads on to boundary analysis and equivalence partitioning.

All this stuff is from math but we don't have, ver often the derivation of where we've taken things in software testing. So we lack something in our definitions. Which means that we have to cram it all into our definition rather than delegate off to someone else 'cause we're an expansive industry that brings thing in.

I thought well there's other complicated things like rocket science. Rocket science is pretty complicated. It must have a really complicated definition. So I asked Google and Google said well its something really very difficult to understand.

You know that's not really that doesn't really explain things.

So we are allowed to have definitions that describe things in ways that make you think, that force you to think about things.

And I thought well I'm really interested in software testing. So Google, what is software testing? And Google doesn't know. Google doesn't know what software testing is. Doesn't have a definition of it in its dictionary.

I'm sure Google has a software process and knows what software testing is. But Google dictionary doesn't have a definition. That's saying to me, search the web. They have no definition, search the web.

Which means we have no choice, and we are essentially having to make this up for ourselves. 'Cause if we search the web what we're seeing, we're seeing the stuff that other people have made up for their selves. Now we can be reliant on what they think or we can make this up ourselves. And then I thought you know, if I was to abstract this and you think what exactly is more important?

Is it imagineedfustication? Or is it working with a good imagineedfusticator right?

In the real world, when you've got something that needs imagineedfusticating. Do you need to know what that is? Or do you need to know someone who can do it?

Is it the definition of imagineedfusticating that is important? Or is it someone who can actually implement it and do imagineedfusting like a good imagineedfusticator.

Now I imagine 'cause I don't know that what you're thinking is what is imagineedfusticating? So I made it up.

Right, it's important that we learn, we get in the habit of making things up. It's a useful discipline. Make up your own techniques, make up your own definitions. Make up your own words! Make up your own concepts.

Now if you're very lucky, some of those concepts will get taken up by other people and you become famous and well paid. And you'll be associated with the concept. No one is going to pick imagineedfusticating. But I called it the art of getting exactly what you need when you need it. Now I would rather work with a good imagineedfusticator, someone who knows exactly what I need and knows how exactly to get it exactly when I need it. Rather than knowing that's what imagineedfusticating is. I want someone to implement that for me.

Most of us in whatever industry we're in are in that position. We are trying to get exactly what someone else needs when they need 'cause that's what makes us valuable.

So software testing is the art of getting exactly what someone needs associated with software testing when they need it. And that's gonna be contextual. That's gonna be dependent on the person we're working with.

So if we assume that we know nothing about software testing, 'cause there's no definitions, we have to search the web. How would we go about working from first principles? 'Cause we are either going to be reliant on somebody else or are we going to make this up. So I'm going to take a worked example.

Here is an application. Now this is an application I, Alan Richardson, have written and you can find it on GitHub. It's a simple CRUD application that manages books and authors and there's drop down menus on the top. It links to go and it's a very simple web app.

I have to test this. Now assuming that I don't know that I developed it, I'm just working from first principles. How do I test this? But if I don't know what testing is, how can I do that right?

Now really I don't want anyone that I don't know how to test it right? 'Cause I'm trying to bluff it.

There's a book by Edgar Rice Burroughs it's called The Efficiency Expert, Pulp novel set in The Depression, guy can't get any work. He goes in to a factory asking for jobs, can't get any work 'cause we've got all the people low skilled people that can do all this job and sees someone else with a clipboard and goes, "What does he do?" He's an efficiency expert, he's helping us make more efficient. So the guy goes around the corner at the 2nd hand book shop finds a manual like how to be an efficiency expert. Starts reading that and then goes in knows enough information to convince someone to give him a job as an efficiency expert. So as long as he stays a couple pages ahead of everyone else he can get well paid. Because it's a Pulp novel, it eventually degrades into fights and romance but the point here being that we can bluff it.

We can learn stuff on the fly and go ahead and do it. We don't necessarily need to know how to start, common sense, a lot of stuff is common sense. Now there are nuances that we will learn over time and experience. But we can start with something simple. So I'm going to gonna go round the corner and find a 2nd hand book shop and find out what is software testing.

Now that's essentially what we're doing on the web. So this is not the text from a web site, it's a text that you might see on a web site and it's representative of the kind of things that people write about software testing.

Software testing is the process of executing a programme or application with the intent of finding bugs in the software. It can also be described as the validation and verification of a software programme or application or product or system.

That's not quite as complicated as the definition I came in. But it's a lot of or's in there. A lot of things we might be testing. So if we simplify this, we're executing something to try to find bugs and that can be described as validation and verification. Well they are big words, I don't know necessarily know off hand what validation and verification is. So I'm going to look that up.

So verification is the process of establishing the truth, accuracy or validity of something. Okay we can call this validation or verification. And verification is related to the validation of something okay?

So validation is the act of checking or proving the validity of something... So validation is validating something. Definitions that are recursive like that aren't particularly helpful. So both of these seem to be quite similar, they're self referential and they're about the truth or accuracy or proving. I'm not going to use those words right? So I have to find things out, this is pretty what this is telling me I have to do stuff to find things out.

And clearly there's someone makes a value judgement on those things. So that's what I'm doing. So I'm gonna forget that but I'm gonna generalise. So software testing is doing stuff to something to find out stuff that I then tell people. So stuff like bugs, what's wrong? What's right? 'Cause we're trying to prove stuff, that's what we got from the other definition. But we have to be careful about certitudes and absolutes.

Find stuff that is wrong. Find stuff that is right. Software testing IS. Clearly if we've got multiple definitions, software testing can be multiple things. What we're working with are descriptions. We have to be careful that we've described software testing and not prescribed software testing. 'Cause if this is software testing this is all that I can do. I might need to go beyond that.

But there is too much theory in there and we can talk about theory forever. We actually got work to do. And we got to try to do this work from first principles, so let's figure that out.

So if I'm thinking what is software testing? I could instead think, what did they mean when they asked me this? Because they've used words and words are communication. And communication is a relationship. So what did they mean? And given my basic knowledge, I can reflect that back to people and look like I know what I'm talking about. So what you want me to do when you test? You want me to find bugs? You want me to verify this? You want me to validate this? How will I know it's a bug? Is there anything specific you want me to test it for? And what do you want to do with what I tell you right? Things might change. What is the end goal with this thing that I give you? Is the testing the input to something else? If so then I might be able to adapt and give you that. What do you mean? Alright, we're throwing this back at people. So that's an important technique in the sense that we're giving somebody else the responsibility for our job.

And if you do this enough times eventually people will just go ahhh too many questions. Alright so here is what I want you to do right? I want you to focus on the create author function, so we got a system that creates authors and books, we're gonna focus on the create author function, make sure it handles a range of input values.

Okay we're bluffing so, "What values?"

I don't know, good stuff, No names, long names, bad names, names, stuff right?

Then we've got if it creates a user, we'll be fine right.

So we've done enough to just annoy them. And we learned stakeholder management, if we were going to put The kind of words that we use to describe software testing on what we just did, we've learned stakeholder management. Needs and requirement analysis. Acceptance condition creation. How will I know that this is correct? Risk based testing, what do you care about? What do you want us to focus on? Test scope restriction, by using the risk, if I'm using the functional scope that we've been given to define it.

We also learned that we can ask questions and people will give us answers and there's nothing wrong with that. Ideally we want to know how to approach this but under certain circumstances we're gonna know tactics, techniques. But they're gonna know the scope so we can ask them.

But I didn't ask, why do you want me to do this?

So why didn't I ask, Why do you want me to test this? That's a rhetorical question. It's partly rhetorical question because this a recording and you cannot answer me back. But as soon as I say why do you want me to test this? If you think about that and you try to answer it, you're gonna come up with some answers. But the reality is you don't know. You don't know why I didn't ask that.

You don't know , you could guess. You're building a model from nothing. You making up assumptions and you're working from your beliefs and that's why I didn't ask why?

Why? is a belief chain question.

If I ask why do you want me to test this, it's a chain of beliefs around testing. Their relationship to testing. Their relationship to me with testing. It's a belief chain, I'm not necessarily getting the scope. I don't know how they're going to answer that. Alright, why do you want me to test this? Because it's your job. Because I don't want to test it. Because testing sucks. Because I want proof that it works right? I don't know how they're gonna interpret that question. Because it's your job that's their interpretation in terms of relationship between me. Because I don't want to test it they're interpreting it between their relationship and them. Because testing sucks, their relationship between the concept of testing. I want proof that works, their relationship of what they are going to do with this information right? I don't want the user to find any problems, their relationship of what is going to happen in the future. Each of these could be a how? or what? when? or who? question. If I ask why, they have to interpret it. And sometimes they will interpret based on beliefs. And if it's beliefs, it can be a chain.

Now this might be useful 'cause we could find the dynamics, we can find out the power structures, we can find out what we need to teach people if they misunderstanding things. We don't have the same understanding of certain concepts that can come across when we ask Why? questions. We might even be able to adopt different view points to test in a different way, all this can be useful. But it might lead to a chain.

Why do you want me to test this? Because it's your job. Why is it my job? Well because that's what we pay for? Why do you pay me for this? That's a good question, 'cause if you don't know what you're doing perhaps we shouldn't, perhaps we should find someone even cheaper right?

You get into a belief chain and your distancing yourself from where that is. Why you want me to test this? 'Cause I don't wanna test this. Why you don't wanna test this? I don't care why they don't want test it, it's irrelevant to what we're doing. But it's a belief chain process. Five Whys is a belief chain process and sometime it's more effective to figure out the path of questioning that we want.

And sometimes when we're dealing with beliefs you can get into conflict. Because people take Why as a challenge, if your challenge their beliefs people try to defend them. So I care about what they believe but I care less about what they believe than more about their needs.

I wanna know what their needs are. How their needs are? When their needs are? Where their needs are? I can't do any imagineedfustication unless I know about their needs. So unless I know when they need them? Unless I know what they need? How they're gonna do that? 'Cause I'm an imagineedfusticator. That's what I want to do.

So what do I do next? Here's the application, go back to the practicalities. I need to create an author. There's the dialogue it says create author, type in the author name and hit the create button and it creates an author. So I'm going to plan it out a bit right? So now I need no name now, I just hit the button. I need a good name. Uh Bob the author seems like a good name, there's probably better names. I can pick real authors, I can use their names. And bad names, Mr. 123 no one has 123. What's a bad name really? Whose defining bad? Is it in terms of naming? Is it cultures, is it... I don't know? What is appropriate in the system, I don't know, bad is too non-specific. I don't know how to do that. But I'm gonna stick with 123 at the moment. So I hit the button and it says please fill in this field.

So I can see that there is some validation when I just hit save. What are the rules? I should check those right? This is something I need to make a note of and do later on. 'cause I've done my data value but I don't know whether that's good enough. So I got questions I need to follow up on. So when I do the good name and create an author and I say Bob the author, it says we've added author Bob the author. Great!

So my work is done or is it?

I mean isn't that is what is suppose to do? How, no one told me what it suppose to do. I'm assuming that's what it suppose to do. How do I know its the correct message, added author Bob the author. Is that correct? I don't know. Do I just decide this? Is there some reference I can use?

And then I'm looking and going wait a minute, is that consistent? 'Cause it says create author and I hit the create button. And then it says we've added it. Should it say created? If it's added, where's added it? What has it added it too?

Oh so many questions and we're building up a model here. Now what we've discovered to put it in software testing terms is the need for an oracle a standard reference something to guide us. Conformance to some standard requirements testing, acceptance condition validation, these are all valid things. But we've also discovered exploration.

We came up with some ideas, we tried them. Our model got expanded, we then have to revisit the plan plan for more things. And I don't think I got enough bad stuff there, I've haven't done bad stuff yet, I've done good stuff. Is it good enough? Revisiting, we're learning as we go. We also created a heuristic.

Look at us, how advanced are we. Not only did we not start with 1970's, 1980's scripted testing. We're way in the current forefront of software testing knowledge, we came up with a heuristic, the consistency heuristic. Is this application consistent with itself? Is this application consistent with other things? The oracles, we are so cutting edge when we're doing this.

But then I'm thinking, well how do I tell people what I've done? I have to report back. So I really better keep some notes that will help me. So I'm gonna start writing down the questions, what things I want to test, what I need to know. How I'm going to track all this? I don't actually know how people gonna do that. So I'm gonna make it up. I'm just gonna start making notes. I'll put them in Evernote, I'll create a list. I'll do something and I'll figure out whether that's good enough. How other people do this?

But we just discovered to put it in testing terms, exploratory testing. The notion of: come up with an idea, doing something, learning from it, planning something else. Planning, test planning. We're building out a test plan for what we're gonna do what we're gonna cover? Why we're gonna do that? What needs are we gonna meet as we do them? What tools do we need? What tools do I need in order to help me do this work? I'm in a browser? I need a tool to make notes.

We're thinking about reporting our work, don't know quite how we're gonna do that yet but we'll adjust it and refine over time. Now the application said the author was added, but did it? Now how can I see that? I need to be able to see that.

I need to be able to interrogate the system to see that its done what it said it did. So I'm gonna look in the list of authors and it says it's there. But can I trust that? Is that good enough?

Now I can see that there's more functionality. I can amend authors, I assume that x is a deletion. I can do stuff, ones a link, ones a button. What's going on... is that OK? I don't know! But I'm learning more stuff. I'm using the application to test itself, but I don't know if I can trust that.

I'm not observing at a low enough level. I don't know how the data is stored. I don't know exactly what data is stored. I don't know when it stored? Where it stored? I don't know how I can see this stored data. Now all of this is simple questioning but it's based around me, I need the feedback that things are working. I need the ability to check that things are happening the way they expect. I need the ability to manipulate it and look at it more detail.

So we've actually just discovered an important characteristic about testing which is Doubt.

How do I know? I haven't got the evidence to back this up. So I doubt what I'm seeing. I need to go in in more detail. But I need to figure out a level at which to stop for that. And the fundamental testing technique that's coming across here is the art of questioning. Asking appropriate questions to get the information we need. I also discovered essentially some graph based testing. I created a thing, I looked in the application to see the information so I got create/read flow. I'm starting to do create, read, update, delete 'cause I saw the amend, I saw the delete. I've got a concept, a model here that I can apply to testing to guide me with my coverage.

So we ask these questions and someone answers it and then right our model just explodes right? Because the validation rules are controlled by HTML using HTML5 using the required and pattern attributes. When the create button is pressed an HTTP POST request the server, ahhh so much stuff.

Now some of you are gonna read that and understand it. Some of you are gonna read that and go I don't understand that technology. So in order to really understand this and test it, we might have to look up this technology. We might have to understand that. But they can see that we're confused so they create a diagram to help us and they walk it through.

Now we got visual representation, I can see paths there. I can see there's a server involved. I need to put things through into the server and that sometimes the server will do validation. But I got stuff I need to learn now 'cause we are continually expanding our model and that's good. We expect this. But it's constant learning.

So I know the concept of HTML, how do I see that? So you show me the Dev tools. And then not just I see concept of HTML, I can see these attributes but I can see there's elements H2, DIV I mean I can kinda map them up and make guesses H2 is bigger than a form which is bigger than just normal text. But I'm kinda drawing conclusions, I need to do more research. But it's interesting that I can see it. Oh my God! Right, I can change it too.

I can edit and delete that required attribute I'm changing the system. Am I hacking? What is happening? How far can I go with this so I can change it? So if I know the validation is done by the required attribute and I delete it. What if there's no validation? I got rid of the validation. So when I hit enter it says, could not find the name to add.

So I know that there is server side validation now. But now I have to test the server side validation and the front end validation. It's gonna take me quite a lot of time to amend the form every time I come in and I want to do server side validation so that we can test the HTTP level then right? 'Cause HTTP traffic is sent, that's what they're telling me, I don't know this. I've just made a statement, it's good enough, I have a problem, and then somebody is explaining things in more detail.

So they tell me about HTTP Debug Proxy where I do something on the front end, the message get sent through a proxy. But now I can work at a proxy level, I can work at the HTTP level, this is a whole new world.

There's a whole new set of information in there that I have to learn my world is just expanding. And getting ever more technical but it's getting technical because I need to do it in order to solve certain problems with the work that I'm trying to do. I'm trying to test more easily and I can do that here. So I ask questions like how does this work? What does this do? How can I see that? How can I make that happen? What should it do? I didn't ask any why? questions, these are very specific questions which got me very specific answers. How? When? Where? That is what is important and knowing how to ask those questions to drill down gets us places faster. Why? might get me there. But it might be a twist and turn and I might ask different people and get different answers.

Now, the interesting thing for me is, this doesn't seem that hard right? We're asking questions but for some people this is a hard thing to do because people don't like to be seen as not knowing something, possibly.

And I say why people find this tough, I don't know, I'm making this up. My beliefs about why other people find this hard are you're reasons why this is hard for you might be different. You're beliefs about other people find this hard might be different this is what I'm assuming. It seems as though when I look online people don't like to say I don't know. They don't like to say that. I don't know why right?

People don't get in the habit of saying I don't know, they bluff it out. Maybe they're concerned that their beliefs don't go far enough and they don't want to be exposed. Or maybe their belief is not set on any rock foundation and if someone quizzes it, it collapses and it got void and that's bad so they don't know.

So what I'm thinking in terms of it, what could these things be? right? Where is questioning is involved right?

We got notions of scientist, skeptics, believers and seekers. Scientist, I'm gonna give you my interpretation of this, they not necessarily dictionary definitions. But scientists for me is someone who is working from a belief model of the world that's backed up by evidence that's reinforced by other evidence from other scientist. And there's an agreed model of the world that people work with. Scientist tends to explore that or just edge slightly outside that belief model but pretty much they're exploring an existing belief model and trying to expand it and check that it continues to hold up under close scrutiny. To really go deep into it. To build the best and most accurate model they can of the world.

Believers. Let's jump to believers. Believers, believe stuff. Almost regardless of evidence right? It doesn't matter if there's, it almost doesn't matter if there's evidence or not. 'Cause it's for them an internal thing. They believe in ... the earth is flat because other people have said it's flat. And it caused them, some say well I mean if it was round, how could it be round right? People in Australia would fall off, how could it possibly be that? It has to be flat, common sense. Look at your table, you drop something off the edge it falls down. People in Australia and it's round they would fall off. Common sense right? So people believe stuff 'cause we can. If it's something outside our realm of checking things, we have to believe people.

We believe scientists. We believe scientists and what they tell us because they done the experiments and they have the authority.

Skeptics tend to straddle the ground between scientist and believers. Scientist don't have time to deal with pseudo scientists who believe stuff that is outside their model. Skeptics tend to bridge that gap. So they will have the scientific model they will ask questions of a believer then they will take that on board quiz them and they go nah you're talking nonsense. But the skeptic has that I will approach over into the world of a believer.

Then we come into the Seeker. Now the Seeker is different. The Seeker is all of these things. Scientist are Seekers when they go outside their models we seek out new things to learn but the seeker has the ability to take on beliefs and discard them later but when they take them on they believe them, they inhabit them they explore the world as though those belief existed. And testers, that's really important right.

Now Ken Campbell was a seeker. Ken Campbell, talked about this a lot he was an almost a improvisational actor. He's a very different acting style. He taught people how to act and explore things. And I he would even be saying things like "well just supposing". Just supposing Fairies did exist. And the seeker would then take that belief fairies exist. Where would fairy exist? If you go out and look in those places. How would I see them if they did exist? How would I know they were there? And that's different from believing and it's slightly different from scientist because we're exploring beliefs which are clearly not scientific and not within any frame of belief. But that belief itself, adopting that belief gives you a different attitude. Gives you a different way of exploring the world. You look at things differently. You will observe things differently. You will try and manipulate different things that might otherwise be there.

Now if projects don't have seekers, projects don't have people trying to go outside the bounds of beliefs of that project. We're not going to find the really hard problems that only occur once in awhile. The edge cases that exist on the periphery of our beliefs systems. Seekers are really important. Robert Anton Wilson someone else I particularly enjoy his books who is also dead. Hopefully this is not a common paradigm.

Robert Anton Wilson said

Belief is the death of intelligence as soon as one believes a doctrine of any sort or assumes certitude, ones stops thinking about that aspect of existence.

Now the danger of this if I believe that quote. If I believe that belief is death of intelligence I might stop thinking about that aspect. But that's okay 'cause I'm a seeker. So I can take that belief, adopt it and discard it right?

Seekers have the ability to take on beliefs, take on different personas. Believe that the system will function under high performance loads we'll focus on somewhere else. Believe that it won't and really hit it hard. Believe that it will handle performance but not when certain things are going on. We will seek out alternative beliefs in order to adopt them. Now given that people have difficulty with this.

I'm gonna give you some tactics just in case, just in case you don't know or if you think not knowing is bad or anything like that.

So this is the summary of it, knowing nothing is good and you don't actually know nothing about it. Are they contradictory? Possibly. Don't care, I'm a seeker. I can have multiple beliefs at the same time. And we ask questions when we know stuff.

So I know nothing about Kardashians, Big Brother all that kind of stuff, reality t.v. and that is good. I would defend the notion that is good, very hard. 'Cause that is a belief but I know nothing about it, I'm comfortable with that. That you can just get comfortable knowing nothing about it is a decision I have made to know nothing about it.

But the reality is right? I don't know nothing about it. I clearly know something, I know their name right? Its just that the things I know about it I've described as nothing right? Like some of the words that we have the definitions we have they're descriptive, they're not prescriptive. And my not knowing about Kardashians doesn't mean that I know nothing about them. It's just describing a lack of something it's as good as nothing. So when we're in this position, what we have is a model that can expand. Its not that I know nothing, it's that I have a model I describe as nothing. So I can ask questions and see if nothing fits that. Do I know nothing about Kardashians, there some women some men there I think. There's, I know some stuff right? By I try to expand my model if I care. right? I try to invalidate parts to my model if I care. Same as normal when we're working with any model. It's not that I know nothing its that I described it as nothing. And our descriptions are not prescriptive. They're attempting to build a model of something.

Now when I'm a manager and I'm working in teams I'm a consultant and I work in teams. I ask questions even when I do know stuff. 'Cause I don't mind if other people think I don't know stuff because I'll say something later on that will boost my credibility again. But I also know that other people in the room don't know.

And people have difficulty saying I don't know. So if I say I don't know then they will learn. I know that people have different understandings right? Because the words are reused in different places so people hear word, we're gonna do a performance test, oh yes, oh yes. But they have completely different models. One person its speed, one person its load. Different models. so they go away with the wrong understanding so I can go, what? I don't understand what is performance testing? and what do you mean by performance testing? Then we will gain more of a shared understanding.

Sometimes it gives someone a chance to share their knowledge by not asking a question, you are not giving someone who wouldn't speak up the opportunity to show their knowledge and develop their skills of communication in a certain environment. And that's important when you're developing your staff asking questions that they can answer. Boosts their confidence and their credibility and their independence of credibility from you. They gain independent credibility as an expert.

'Cause asking questions fundamentally is a good thing to do. And that's what testers do. Testers are seekers. Testers bring in the attributes of the scientist, the sceptic, the believer all in one in the seeker 'cause we can take on beliefs and we can see the value in doing something even when no one else does. And we can drop that belief and bring in another one.

Seekers is what we're attempting to do because fundamentally asking questions is good and it's how we explore the world. And build up models from nothing, or what we describe as nothing which is common sense.

Some Notes and Thoughts on Risk

I'm just finishing up at Eurostar. In fact, I've finished the Eurostar. I'm just waiting for the taxi to arrive, and then I'll head off.

One of the things that came up when I was speaking to other speakers was the topic of risk because it seems though a lot of certainly the speakers I was speaking to think that risk has lost prominence within testing, that people don't think about risk, particularly in Agile projects.

I don't know if that's true or not, and certainly I think about risk pretty much all the time, and when I'm on projects and consulting, I'm expressing risk, and not just business risk, but process risk, tool risk and technical risk in terms of the architecture, and you can pretty much find risk anywhere. And I think that risk attitude has a knock-on effect on like just my daily life.

It's one of those things that impacts how you interact with the real world. Like your technical skills and the web impact how you interact with the web. Risk analysis impacts how you deal with the real world.

So in the hotel when I came in, I was checking to see where are the stairs 'cause I was only on the first floor, so using the stairs is a lot easier. I have to go down the fire escape exit, but I can still go up the stairs. I checked out last night rather than this morning in case there's a queue for the taxi.

I got up early this morning to check my emails to make sure the taxi was confirmed, which it wasn't, so I then had to go down to the hotel lobby to get a taxi to take me to the airport. I leave early enough so that if there's traffic, I don't miss the plane.

I've got a contingency plan that I prepared before I came here about how I can get to the airport if the taxi system all failed.

So that's, for me, risk is one of the most important things that we deal with, and if that's not part of what you do, I thoroughly recommend that you explore risk 'cause it's something we should not be forgetting about.

Lightning Talk - Our Agile Process is Screwed Up

So now I don't have to get struck by lightning, Fran's shown you what happens.

So I'm gonna follow on from that.

All right, 'cause I get asked similar questions.

I get asked "Dear Evil Tester". Emails come into me, "our Agile project is screwed up and messed up, what can I do?"

All right, 'cause people want to fix it, people want to take responsibility for that.

Where screwed up means some of the things that Fran just talked about.

Where we've got no time for testing... by the testers.

We've got no time for automating... by the testers.

We've got no acceptance criteria... that the testers can use.

We've got no story details... that we as testers can understand, we're not invited to meetings, and so the tester's not being included, we're not getting stuff we want.

And the fundamental problem is that you're not actually Agile.

You're not actually doing an Agile project.

You've called it Agile, you've labelled it Agile, you've got some ritual that you call Agile, you do stand-ups, you do retrospectives but you don't learn from them.

The feedback doesn't come in as input and you process it.

So what we have to do then is we still have to fight the good fight and explain what Agile is and try and do the Agile thing.

But we still have to work with the actual reality on the ground so there are steps that people can take.

The first thing we have to do is we have to get our own house in order. Make sure that we know how to test things, make sure that we are avoiding waste in the testing that we do. So that we're not writing big test plans, we're not writing huge amounts of stuff in advance, we're keeping everything lean so that we can respond effectively.

We need to make sure that we do own the things that are ours to own. So exploratory testing, that we make sure that we do that. That we do that properly, we capture all the results and we plan it in advance.

One of the things you can do is you can use the Agile words against the fake Agile process.

So Agile has words like Done, Not Done.

When you're planning out your testing, your exploratory testing, and build charters, and whatever lightweight plan you want to do, you say "not done", then you list things that you want to do.

Then when you start doing them you gather evidence you say "done", then you put them into a Done list.

So then at the end when people try and go, "can we close this story now? can we mark the story as done?", in their face there's feedback, that it is not yet done, 'cause there's a bunch of stuff not done and you're making it visible.

You're helping people understand and share the pain that you've got.

The other thing we have to do, and this is, why we get in a mess because in Agile you are supposed to do the work of other people as well, you are supposed to absorb things.

But that's what we have to not do, you have to focus on the work that we do, and not absorb the poor output from other things.

So testers do not stay late testing because the story was delivered at four o'clock in the afternoon, while everyone else goes home.

If we absorb things, then we help this bad process continue and in a normal Agile process you would want to help out, you want to make it work, but you're not in an Agile process.

We have to work with the reality, and that means making people share the pain, making this stuff visible. We have to give them a reason to want to change, because the system has to change itself.

We can only act on small things.

Thank you very much.

Lightning Talk Practice with Additional Information about Cybernetics and feedback

So I'm at Eurostar, I was thinking about what lightning talk I'm going to do this morning.

And yesterday and a couple of days over the conference I've been getting certain questions coming through. So what I'm going to talk about is what testers can do and an easy way on Agile projects to help themselves. I will stick the timer on for four minutes, I'm going to work from my notes and we'll see how far I get.

So generally what I'm asked is, "Our Agile project is screwed up. What can we do?" And I do get asked this a lot. LinkedIn, various other places, a couple people at the conference have asked. And what screwed up means, we don't have any time for testing. By the testers. We've got no time for automating, again by the testers. People are not skilled enough doing the work, we don't know how to test this. We've got no acceptance criteria. That's good enough for the testers. There's no story and details, we don't invited to meetings. And again this is all tester related.

It's not that these things aren't happening, it's just everything here is related to the test part.

And the problem here is that the projects themselves are not Agile. They're calling themselves Agile. They use Agile words, they have Agile rituals in there: retrospectives, stand ups and three amigos whatever else but there's no agility in there. There's no attempt to continually improve and refine the system. So we have roles and subsystems and testing, anything with testing it is done by the testers.

This is a problem.

Now, we have to keep fighting the good fight which is explaining what Agile is, explaining what testing is within Agile. But we also have to deal with the actual reality we face.

So some tactics for doing that are first of all, control the things that you have control over. Right, so get your own house in order. Make sure that you have the skills to do your testing. You've set the expectations you know what expectations people have and you've agreed them. You avoid waste in your process. You're not writing big test plans. You keep things as lean as possible. Keep things documented as lightly as possible, at the point that you need it. Keep it lean, and keep control.

Own the things that are yours to own. So own exploratory testing, and do that well. Make sure you do your exploratory testing, you write down your plan of what you're gonna attack. That way, if you use Agile words like done and not done, when you've got a plan. When people come to say "This story is done." and see that your testing has got all the stuff that's done with all the evidence and all the steps that you took. But it's also got these things that are not done, 'cause you haven't had time.

It's harder for them to ignore that there's a problem. 'Cause that feedback is staring them in the face, and they have to act on it. So make sure that you're writing down the things that show the scope, that show the effort, that show the work.

Make the testing visible, but not just make the testing visible make the testing visible such that it acts as negative feedback in the process, so they know where the pain points are for testing. And this one is key and slightly more controversial. Which is, don't do the work of others. Right, 'cause very often what happens is if you give an inch they'll take a mile and they'll keep taking the mile.

So, if you don't have enough time to test, that the software story is delivered at the end of the day the testers might stay late in order to test it. To provide the feedback so that programmers can work on it in the morning. Don't do that. Right, you're doing somebody else's work. You're absorbing their failure in the system and taking it on board.

That way they have no pain. They've got no negative feedback causing them to change the process.

Do not, absorb the work of others. Don't do the work of others.

If we've agreed as a team, that everyone is going to do automated execution and build test code, if that's what we're calling it, then don't do that on your own. Bring people in and keep reminding people that we said we were going to pair on this, can we pair on this now, can we do this now.

Keep pushing for that, make opportunities for it to happen but control your part of the process. You don't attempt to fix Agile, you attempt to fix the bit that you were involved in.

So that's four minutes. So I didn't quite cover everything I've got written down. Other things I've got written down, not much more.

Part of the issue is, when we absorb other people's parts of the process we then can look bad because we are doing what we think it takes to make the project keep going on, but we're getting enough time to do everything that we want to do.

So we're not getting a chance to do all the testing that we want to do because we're trying to do some automating as well. We're not getting the chance to do all the automating because we're doing the testing. We're refining the stories late so we're having to go have meetings about the acceptance criteria to figure out what we need to test after the story has been played. Which then cuts into time that we could be using for testing, or automating.

And the fact that we're absorbing all this makes the project look good. Makes the project run more smoothly, which is important. But it makes us look bad, and makes testing look bad on the project because we're trying to do our best.

And sometimes we have to let the system fail, by fitting in with sustainable pace. Whatever is its 37.5, 40 hour weeks. To keep things on track, but to make visible what we have done and not done.

'Cause it's negative feedback that causes change.

I was thinking yesterday, that a lot of this is coming from cybernetics when I think about it. In terms of, we talk about feedback a lot in Agile. So we have retrospectives, we have standups. We have lots of interface points that we're looking at feedback in order to come in. But then I realised yesterday, it's feedback in the sense that it's an output that comes in as input again. But, if the input is not processed it makes no difference. We say we're going to do certain things we have these goals that comes in as feedback.

If we don't act on it, it's irrelevant. We've filtered it out. It's not impacting the process. It doesn't cause the process to change. Which is why in retrospectives you don't just come out with goals. You come out with the next steps.

And you do the next steps.

And then you can remind yourselves during the actual sprint about those next steps. Have we done this?

You can do that in the standups. When are we going to do this? Do you have time today to do this?

You can convert them into stories to make sure that you do them 'cause they're on your board. There's steps that we can take to make things visible, and make it actionable.

That systems, only change when they have negative feedback coming in that's processed. Very often what we have is negative feedback that comes in that is filtered out or, and this is what often happens to a tester, absorbed by a single part of the system.

So negative feedback comes in, that we don't have enough time to do automating. That gets absorbed by the test part, so no one else gets involved in the automating. Test part then has to work longer hours or weekends to try and get this done. Or it becomes testing stories that never

get finished, they look back, they look bad that part gets absorbed by the testing part. And it shouldn't.

An effective system doesn't have subsystems absorbing negative feedback. Particularly when it's system feedback. And most of the feedback on an Agile project is system level 'cause it's the system that has to change, it's the way that we develop. It's the way that we work as a team, that has to change. And until that feedback get absorbed as the system, the process doesn't change. Which is why its hard for individuals to fix it because individuals take that feedback and absorb it on their own.

Now ideally, in a four minute talk I would like to mention that as well but then people have to understand the notions of cybernetics. And I'm concerned that people might not understand cybernetics or feedback. So I'm going to talk about it more general clear actions steps you can do. To me that's coming from the notion of cybernetics and systems viewpoint of how people work.